Portfolio Summary

Student: Abdulrasheed Abdulrazak
Matriculation number: 2026/A/MIT/0734
Student ID: 301926758
Course: MIT 8103 Advanced Database Systems
GitHub repository: https://github.com/bigphysiology-coder/advanced-database-portfolio

---

1. Case Study

Riverbend Teaching Hospital is a simulated 500-bed tertiary teaching hospital providing outpatient, inpatient and laboratory services. The database is its **Clinical Records System**: the system of record for who was seen, by whom, for what, and with what result. The same case study is used in all five portfolios. The data is synthetic, generated by this repository from distributions modelled on published hospital volumes; no real patient data is used. The working dataset contains 20,000 patients, 82,000 appointments, 62,588 encounters, 125,176 prescriptions, 93,885 laboratory orders and 163,387 results. The full statement is in `docs/case-study.md`.

2. Portfolio 1 — Database Design and Modelling

The result: a 21-relation schema normalised to 3NF, with four documented denormalisations and all 10 negative constraint tests passing. It separates patient identity, scheduling, clinical encounters, prescribing, laboratory work and audit history. See `er-diagram.png`, `schema.sql` and `normalisation.md`.

Three decisions shaped the model. First, `patient` uses a UUID surrogate primary key plus a unique MRN business key, so an MRN correction touches one row instead of cascading through appointments and laboratory orders; the cost is one extra join when a human-readable identifier is needed. Second, safety rules live in the database: an `EXCLUDE USING gist` constraint prevents overlapping appointments, CHECK constraints reject malformed or impossible values, and an append-only trigger blocks `UPDATE` and `DELETE` on `audit_log`. Third, the four denormalisations are deliberate and documented, including JSONB audit snapshots that stay readable after schema changes. The seed was rescaled from 500 to 20,000 patients because plans were indistinguishable at the smaller size; the database is 312 MB.

3. Portfolio 2 — Query Processing and Optimisation

The patient-history query fell from 197.5 ms to 0.60 ms (330×) after three indexes; the double-booking audit fell from 646.2 ms to 55.9 ms with no new index. The after timings are medians of a 21-run capture; the first run is discarded. Plans, buffers and incremental measurements are in `execution-plans.md`.

Q1 was repeated work, not a missing index: two correlated subqueries re-scanned 125,176 prescriptions and 93,885 laboratory orders per encounter. The encounter index alone moved 197.5 ms to 178.2 ms; the prescription index took it to 82.0 ms; the laboratory-order index took it to 0.78 ms. Q6 was the opposite lesson: a range self-join probed the GiST exclusion index 11,800 times and read 57,300 buffers to return zero rows. A window over one ordered pass reduced the buffers to 294 and the median to 55.9 ms. A candidate B-tree measured 29–72 ms (median about 44 ms) against 43.9 ms for the rewrite alone, so it was rejected as write overhead. The Q5 `SELECT DISTINCT` rewrite was also rejected: it was faster without an index but blocked the covering-index plan, producing 871 ms against 303.4 ms for the original query with the index. The accepted indexes occupy 20.4 MB (6.5%). Q4 is the honest counter-example: its predicate selects 85% of the table, so its 1.5× gain is close to the ceiling.

4. Portfolio 3 — Transactions and Concurrency

The unprotected booking race produced two appointments in one slot; the exclusion constraint rejected the second with SQLSTATE `23P01` after a 1.844 s wait, and a serializable on-call update aborted with `40001` rather than leaving zero doctors on duty. Raw two-session captures, monitor samples and exit codes are in `portfolio-3-transactions-concurrency/evidence/`.

The live constraint was not dropped. A structurally identical `p3_demo.booking_unprotected` table with the same application logic showed both sessions reading zero rows and both committing — the classic check-then-act race. The exclusion constraint rejects the second writer. A pessimistic `SELECT ... FOR UPDATE` also works (1.913 s wait) but serialises every booking for that doctor, so it is demonstrated, not adopted.

Isolation was tested separately: the same count read twice moved from 5 to 6 under `READ COMMITTED` and stayed at 5 under `REPEATABLE READ`. That protects a single transaction's reads but not a multi-row invariant. Two doctors each took themselves off duty after seeing the other on duty, and both committed under `READ COMMITTED`, leaving zero on duty. Under `SERIALIZABLE`, the second transaction aborted with `40001` and one doctor remained. The design therefore uses the exclusion constraint for appointments and `SERIALIZABLE` for the on-call invariant, where no exclusion key exists.

5. Portfolio 4 — NoSQL and Advanced Data Models

The laboratory store holds 93,885 order documents with 163,387 embedded results; the `$jsonSchema` validator passed 11/11 cases and the cross-store consistency suite passed 27/27 checks. See `model-design.md` and `portfolio-4-nosql/evidence/`.

PostgreSQL remains the system of record for patients, encounters, staff, appointments, prescriptions, the laboratory order header and the audit log, because those entities need foreign keys, exclusion constraints and multi-row transactions. MongoDB stores one document per laboratory order with its 1–4 result analytes embedded. That array is bounded (average 2.49 results, maximum 4), the document is read and written as a unit, and the report must preserve the units and reference ranges in force when it was produced. Patient and encounter references are UUID strings; there is no cross-store foreign key, so a derived `patient_refs` projection and the consistency checks make drift visible.

The benchmark refuses the usual NoSQL claim. Warm PostgreSQL matched or beat MongoDB on patient history (2.07 ms vs 8 ms) and analyte trend (2.8 ms vs 7 ms); the dashboard comparison is not like-for-like (96 ms for one PostgreSQL branch vs 3.24 s for MongoDB's four-branch `$facet`). MongoDB won the laboratory work queue (8 ms vs 26–35 ms) and the single-document write (16 ms vs 122.9 ms). The document model was chosen for shape, ownership and write unit, not raw speed; for a small single-node hospital, a PostgreSQL `jsonb` column would be simpler. The cost is ETL, eventual consistency and no cross-store foreign key.

6. Portfolio 5 — Distributed and Cloud Database

A PostgreSQL 16 primary and hot-standby replica measured 525.469 ms of commit-to-visible lag after a 50,000-row burst; a paused-replay read returned zero rows while 296 bytes of WAL were unapplied, and the lag-aware router sent that read to the primary. The deployment, configs and raw captures are in `portfolio-5-distributed-cloud/`, with the trade-off analysis in `architecture.md`.

Asynchronous streaming replication is the default because a synchronous commit would make every booking wait for the standby, and a standby outage would stop clinical writes. The cost is a non-zero RPO, stated rather than hidden. Synchronous mode was demonstrated with `synchronous_standby_names`, producing `sync_state=quorum` and a median commit of 16.913 ms against 6.971 ms asynchronous. Manual promotion completed in 2.9467 s and lost zero rows, but only because the replica was deliberately caught up first; that controlled result is not evidence that asynchronous RPO is zero. Sharding was rejected at 317 MB and 20,000 patients. If the hospital grew to regions, the natural shard key would be `(region, hospital_id)`, and patient moves would be the hard cross-shard transaction. The single-host demo does not prove automatic failover, multi-region availability or PITR.

7. Technical Reflection

The main lesson was to measure the mechanism rather than trust its label. I assumed that `synchronous_commit=on` made replication synchronous; the primary still reported `sync_state=async` until `synchronous_standby_names` was set. The same pattern appeared in Q6: my first window rewrite used `lag(slot_end)`, which misses a long appointment overlapping an intermediate row, so I replaced it with a running maximum over all earlier rows. Both errors were caught by reading the output rather than the intention. The full first-person reflection is in `reflection.md`.

8. References

1. PostgreSQL Global Development Group. (2026). *PostgreSQL 16 Documentation* (16.15): High Availability, Load Balancing, and Replication; Synchronous Replication; `pg_basebackup`; `pg_stat_replication`; `EXPLAIN`; Transaction Isolation; `CREATE INDEX`. https://www.postgresql.org/docs/16/
2. MongoDB Inc. (2026). *MongoDB 7.0 Manual* (7.0.14): Schema Validation, Indexes, Aggregation Pipeline and `$merge`. https://www.mongodb.com/docs/manual/
3. Docker Inc. (2026). *Docker Desktop Documentation*. https://docs.docker.com/desktop/
4. Elmasri, R. & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.
5. Silberschatz, A., Korth, H. F. & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill.
6. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly.
7. Dataset: synthetic data generated by `seed-data.sql`. No external patient dataset was used.

9. AI Use Declaration

AI assistance is declared in full in `docs/ai-use-declaration.md`. OpenCode (DeepSeek V4.1 Flash Free) was used between 25 and 30 October 2026 to draft SQL, scripts and documentation. Every output was run against PostgreSQL 16.15 or MongoDB 7.0.14, checked against raw captures, and corrected where it failed. The declaration records the specific corrections, including the synchronous replication assumption and the Q6 window bug.